Node.js Routing Examples

CCSW-321 WEB DEVELOPMENT

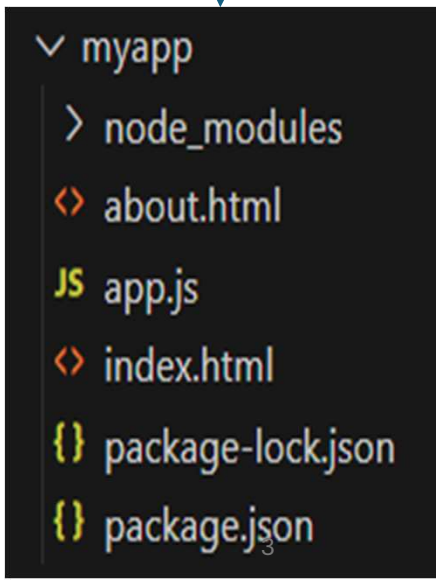
Static Routing

Example 1

How to use Routes with serve-static Files in Node.js ?

Using serve-static files with routes in a Node.js application involves creating a server with Express, defining routes, and serving static files like HTML, CSS, JavaScript, images, etc. Here's an article explaining how to use serve-static with routes in Node.js.

You create a Folder myapp that will include all the files



Serving Static Files with Routes in Node.js

When building web applications with Node.js, you often need to serve static files like HTML, CSS, JavaScript, and images. The serve-static middleware in conjunction with Express is a popular way to achieve this. In this article, we'll walk through the process of using serve-static to serve static files alongside custom routes in a Node.js application.

Approach

To use routes with the **server-static** files:

•Use **http** to create the server and **fs** to handle file operations, defining the server port for connections.

•Implement a **readAndServe** function to read files asynchronously and **send the content** as an **HTTP response**.

•Set up **http.createServer** to check the request URL and call **readAndServe** with the appropriate **file path for each route**.

•Use **server.listen** on a specified port to begin listening for incoming requests and log a message confirming the server is running.

•Develop **index.html** and **about.html** with basic content and navigation links, placing them in the same directory as the server script.

Example: ***Implementation to use routes with serve-static files in NodeJs.***

```html
<!-- index.html -->
<html>
  <head>
    <title>index</title>
  </head>
  <body>
    <h2>Welcome To this Routing Example</h2>
    <p>This is Index file</p>
    <p><a href="/about"> Click to go to About Page </a>
    </p>
  </body>
</html>
```

```html
<!-- about.html -->
<!DOCTYPE html>
<html>
  <head>
    <title>About</title>
  </head>
  <body>
    <h2>Welcome To About Page</h2>
    <p><a href="/"> Click to go to index </a>
    </p>
  </body>
</html>
```

```
// app.js
const http = require('http');
// File system module used for accessing
files in nodejs
const fs = require("fs");
const port = 3000;
// Helper function
function readAndServe(path, res)
{
    fs.readFile(path,function(err, data)
    {
        console.log(data);
        // res.setHeader('Content-Type',
'text/html');
        res.end(data);
    })
}
const server = http.createServer((req, res)
=> {
  const url = req.url;
  const method = req.method;
  /* Serving static files on specific Routes
*/
  if(url === "/")
  {
      readAndServe("./index.html",res)
}
```

```
/* The file named index.html will be sent
        as a response when the index url is
requested */

else if(url === "/about")
  {
      readAndServe("./about.html",res)
      /*The about.html file will be sent as a
response when
        /about is requested */
  }
  else
  {
      res.end("Path not found");
      /* All paths other than / and /about will
send an error as
      a response */
  }
});
server.listen(port, () => {

  console.log(`Server running at
 http://:${port}/`);
});
```

Static Routing

Example 2

In Node.js, particularly when using the Express.js framework, **static routing** refers to **serving static files** like HTML, CSS, JavaScript, images, and other assets directly from a designated directory. This is typically achieved using middleware, most commonly **express.static.**

Here's an example demonstrating static routing in a Node.js Express application:

```
const express = require('express');
const path = require('path'); // Node.js built-in path module
const app = express();
const PORT = 3000;

// Serve static files from the 'public' directory
// Any request to a URL that matches a file in 'public' will be served directly.
// For example, if you have 'public/styles.css', it will be accessible at /styles.css
app.use(express.static(path.join(__dirname, 'public')));

// You can also add a route prefix for static files
// app.use('/assets', express.static(path.join(__dirname, 'public')));
// With this, 'public/styles.css' would be accessible at /assets/styles.css

// Define a simple route for the homepage
app.get('/', (req, res) => {
res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

// Start the server
app.listen(PORT, () => {
console.log(`Server is running on http://localhost:${PORT}`);
});
```

To make this example functional, you would need the following file structure:

```
your-project-folder/
├── app.js // The Node.js application file
└── public/
├── index.html
├── styles.css
└── images/
└── logo.png
```

Explanation:

- `const express = require('express');:`

- **Imports the Express.js framework.**

- `const path = require('path');:`

- **Imports the `path` module, which is useful for constructing file paths correctly across different operating systems.**

- `app.use(express.static(path.join(__dirname, 'public')));:`

- **This is the core of static routing.`express.static()` is built-in middleware in Express for serving static assets.**

- `path.join(__dirname, 'public')` constructs the absolute path to a directory named `public` located in the **same directory as your main application file. This `public` directory will contain your static files (e.g., `index.html`, `styles.css`, `script.js`, `image.png`).**

- `app.use()` **mounts this middleware. When a request comes in, Express will first check if the requested URL matches a file within the specified static directory. If a match is found, that file is served directly.**

- `app.get('/', (req, res) => { ... });:`

- **This demonstrates a standard route handler for the root URL (/). This is distinct from static file serving, as it involves executing a function to generate a response, rather than simply serving an existing file.**

To use this example:

- **Install Express:** If you haven't already, install Express in your project: **npm install express**

- Create a `public` directory:
- In the same directory as your main Node.js file (e.g., **`app.js` or `server.js`**), create a folder named `public`.
- **Add static files:**
- **Place your static files (e.g., `index.html`, `styles.css`) inside the `public` directory.**
- For example, create **`public/index.html`**:

```
<!DOCTYPE html>
<html>
<head>
<title>Static Page</title>
<link rel="stylesheet" href="/styles.css">
</head>

<body>
<h1>Hello from a static HTML file!</h1>
<script src="/script.js"></script>
</body>

</html>
```

And public/styles.css.

```
body {
font-family: sans-serif;
background-color: lightblue;
}
```

Access in browser: Open your web browser and navigate to `http://localhost:3000`. You should see the content of `public/index.html` served. You can also directly access static assets like `http://localhost:3000/styles.css`.

HTML Routing

Example 1

The **express.urlencoded()** middleware in Node.js with Express is used to parse incoming requests with URL-encoded payloads, typically from HTML forms submitted with the application/x-www-form-urlencoded content type. It populates the **req.body** object with the parsed data.

Here is an example demonstrating its use:

```
const express = require('express');
const app = express();
const port = 3000;

// Middleware to parse URL-encoded form
data
app.use(express.urlencoded({ extended: true
}));
```

To Include This part here

```
app.listen(port, () => {
console.log(`Server running at
http://localhost:${port}`);
});
// Serve an HTML form
```

```
app.get('/', (req, res) => {
res.send(` <!DOCTYPE html> <html> <head>
<title>Form Example</title> </head> <body>
<h1>Submit Your Information</h1> <form
action="/submit" method="POST"> <label
for="name">Name:</label> <input type="text"
id="name" name="name"><br><br> <label
for="email">Email:</label> <input
type="email" id="email"
name="email"><br><br> <input type="submit"
value="Submit"> </form> </body> </html> `);
});

// Handle form submission
app.post('/submit', (req, res) => {
const { name, email } = req.body; // Access
parsed data from req.body
res.send(`Thank you, ${name}! Your email is
${email}.`);
});
```

Explanation:

•app.use(express.urlencoded({ extended: true }));:

•This line integrates the express.urlencoded() middleware into your Express application.extended: true option allows for parsing of rich objects and arrays encoded in the URL-encoded format, using the qs library. If set to false, it uses the querystring library and only allows simple key-value pairs where values are strings or arrays of strings.

•app.get('/'):

•This route serves an HTML form to the client when they access **the root URL**. The form's action attribute is set to /submit and its method is **POST**.

•app.post('/submit'):

•This route handles the **POST** request when the form is submitted.Because express.urlencoded() is used as a global middleware, it parses the incoming **URL-encoded** data from the form submission before the route handler is executed.

•The parsed data is then available in req.body as a JavaScript object, where the keys correspond to the name attributes of the form fields (e.g., name and email).

•**The route then uses this data to send a personalized response**.

HTML Routing

Example 2

How express.urlencoded Works?

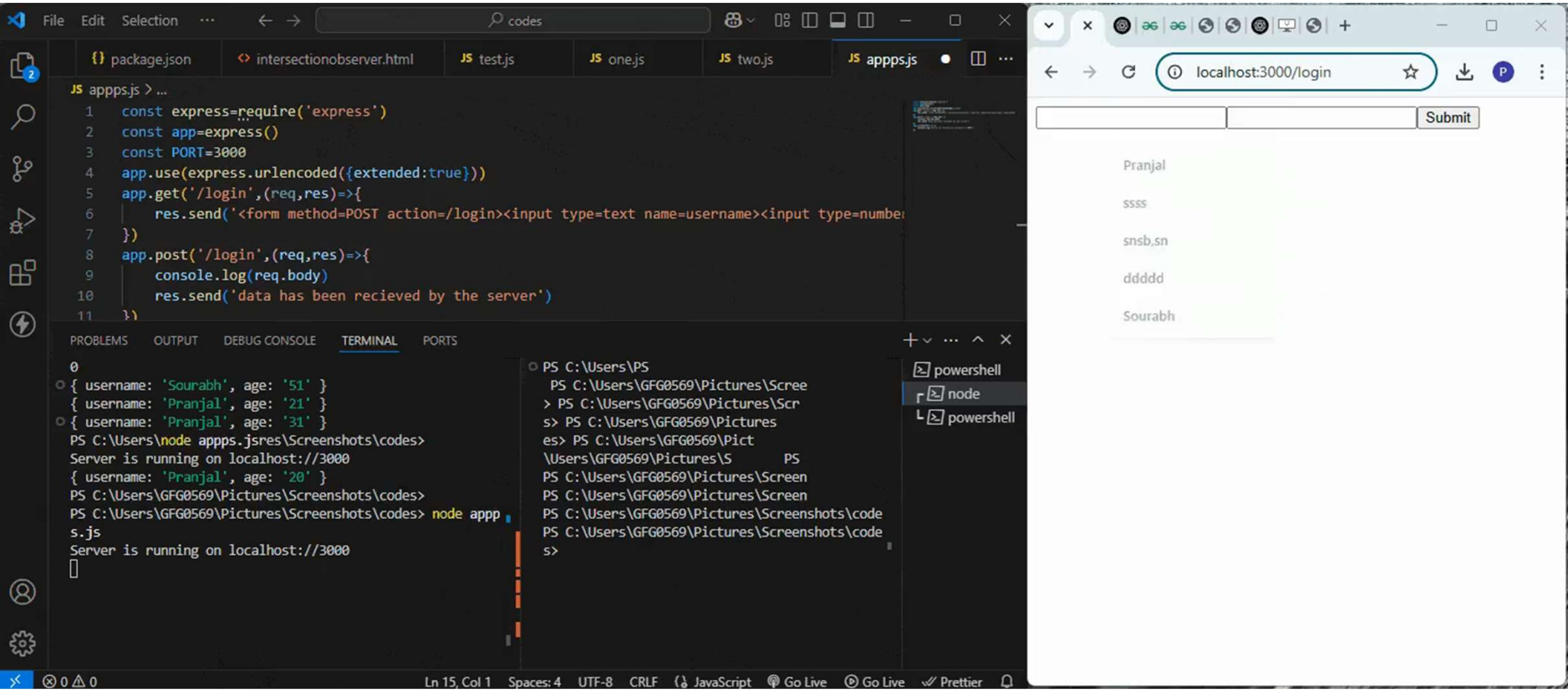
The express.urlencoded() middleware processes URL-encoded form data, converts it into a JavaScript object, and makes it available via **req.body**.

1. First it processes form data sent via POST requests with application/x-www-form-urlencoded encoding.
2. Then it converts the incoming data into a JavaScript object and makes it accessible through **req.body**.
3. When { extended: true } is set, it supports nested objects and arrays using the **qs library**.
4. It includes security options like limit to restrict body size and **parameterLimit** to prevent excessive form fields.
5. It only parses requests with Content-Type: application/x-www-form-urlencoded, ignoring other data formats.

Now lets understand express.urlencoded with another example

```
const express = require('express')
const app = express()
const PORT = 3000
app.use(express.urlencoded({ extended: true }))
app.get('/login', (req, res) => {
    res.send('<form method=POST action=/login><input type=text
name=username><input type=number name=age><input type=submit></form>')
})
app.post('/login', (req, res) => {
    console.log(req.body)
    res.send('data has been recieved by the server')
})
app.listen(PORT, () => {
    console.log('Server is running on localhost://3000’)
})
```

Output (Video)



Explanation:

- **Imports and Sets Up Express**: The code imports Express, initializes an app instance, and sets a server port (3000).
- **Middleware for Form Data Parsing**: app.use(express.urlencoded({ extended: true })) is used to parse URL-encoded form data and make it available in req.body.
- **GET Route for Login Form**: When a user visits **/login**, a simple **HTML form** is displayed, allowing them **to enter a username and age**.
- **POST Route to Handle Form Submission**: When the form is submitted, the server **logs the received data (req.body)** and sends a response confirming receipt.
- **Starts the Server**: The app listens on port 3000, and logs Server is running on **localhost://3000** when it starts.

When using the express.urlencoded

•**Handling User Authentication Forms**: Used to process login and signup forms, where users submit credentials like username and password via a POST request.

•**Processing Contact or Feedback Forms**: When users submit a contact form on a website, **express.urlencoded()** parses the form data (e.g., name, email, message) and makes it accessible in req.body.

•**Submitting Order Forms in E-commerce Apps**: When a customer fills out an order form, the middleware extracts details like product name, quantity, and address for order processing.

•**Storing Survey or Questionnaire Responses**: Used to collect and store survey responses, where users submit multiple-choice or text responses through a form.

•**Handling Profile Update Forms**: In user dashboards, when users update their name, bio, or preferences via a form, express.urlencoded() ensures the server correctly processes the updated data.

JSON Routing

Example

Here is an example of JSON routing in Node.js using the Express framework:

1. Project Setup:

Create a new directory (**json-routing-example**) for your project and initialize a Node.js project:

2. server.js (Main Application File):

Create a **server.js** file and set up your Express application:

```
const express = require('express');
const app = express();
const port = 3000;

// Middleware to parse JSON request bodies
app.use(express.json());

// Import your routes
const userRoutes =
require('./routes/users');
const productRoutes =
require('./routes/products');

// Mount the routes
app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);
```

```
// Basic root route
app.get('/', (req, res) => {
  res.send('Welcome to the JSON
Routing Example!');
});

app.listen(port, () => {
  console.log(`Server listening
at http://localhost:${port}`);
});
```

3. routes/users.js (User Routes):

Create a routes directory and a **users.js** file inside it to define user-related routes:

```
const express = require('express');
const router = express.Router();

let users = [
{ id: 1, name: 'Mohammed', email:
'Mohammed@uj.edu.sa },
{ id: 2, name: 'Kais', email:
'Kais@uj.edu.sa' },
];

// Get all users
router.get('/', (req, res) => {
res.json(users);
});

// Get user by ID
router.get('/:id', (req, res) => {
const id = parseInt(req.params.id);
const user = users.find(u => u.id === id);
if (user) {
res.json(user);
} else {
res.status(404).json({ message: 'User not
found' });
}
});
// Create a new user
router.post('/', (req, res) => {
const newUser = req.body;
newUser.id = users.length > 0 ?
Math.max(...users.map(u => u.id)) + 1 : 1;
users.push(newUser);
res.status(201).json(newUser);
});
module.exports = router;
```

4. routes/products.js (Product Routes):

Create a **products.js** file in the routes directory to define product-related routes:

```
const express = require('express');
const router = express.Router();

let products = [
{ id: 101, name: 'Laptop', price: 1200 },
{ id: 102, name: 'Mouse', price: 25 },
];

// Get all products
router.get('/', (req, res) => {
res.json(products);
});
```

```
// Get product by ID
router.get('/:id', (req, res) => {
const id = parseInt(req.params.id);
const product = products.find(p => p.id ===
id);
if (product) {
res.json(product);
} else {
res.status(404).json({ message: 'Product
not found' });
}
});

module.exports = router;
```

5. Running the Application:

Start your Node.js application: **node server.js**

Explanation:

•express.Router(): * (See next slides for further explanation)**

•This creates modular, mountable route handlers. Each router instance is a complete middleware and routing system.

•app.use(express.json()):

•This middleware parses incoming requests with JSON payloads, making req.body available for POST/PUT requests.

•app.use('/api/users', userRoutes):

•This mounts the userRoutes router at the /api/users base path. Any routes defined in userRoutes will be prefixed with /api/users.

•res.json():

•This method sends a **JSON** response to the client with the correct Content-Type: application/json header.

•req.params.id:

•This accesses route parameters (e.g., the id in /api/users/:id).

This setup demonstrates how to organize your routes into separate files for better maintainability and how to serve JSON data from different endpoints in a Node.js Express application.

***express.Router() in Node.js with the Express framework creates a new router object that functions as a "mini-application." This object allows for modular and organized handling of requests and routes within an Express application.

Purpose and Benefits:

•**Code Organization:**

•It enables the separation of routes into different files based on functionality or resource (e.g., users.js for user-related routes, products.js for product routes). This improves maintainability and readability, especially in larger applications.

•**Modular Routing:**

•Each router instance can have its own middleware and route handlers, operating independently of the main app instance until it's mounted.

•**Cleaner Main Application File:**

•By moving route definitions to separate router files, the main app.js or server file remains cleaner and focuses on global middleware and mounting the routers.

How to Use express.Router():

•**Create a Router File:** In a separate file (e.g., `routes/users.js`), require Express and create a new router instance:

```
const express = require('express');
const router = express.Router();
```

•**Define Routes and Middleware:** Add route handlers and middleware specific to this router using methods like `router.get()`, `router.post()`, `router.use()`, etc.:

```
// Middleware specific to this router
router.use((req, res, next) => {
console.log('Time:', Date.now());
next();
});

// Define routes
router.get('/', (req, res) => {
res.send('Users home page');
});

router.get('/:id', (req, res) => {
res.send(`User profile for ID: ${req.params.id}`);
});
```

•**Export the Router:** Make the router available for import in other files:

```
module.exports = router;
```

•**Mount the Router in the Main Application:** In your main app.js file, import the router and mount it to a specific base path using **app.use():**

```
const express = require('express');
const usersRouter = require('./routes/users'); // Import the router
const app = express();

app.use('/users', usersRouter); // Mount the router at the '/users' base path
```

Now, any requests to /users or /users/:id will be handled by the routes defined in **usersRouter.**